🟢 Aula 6 — Ciclo de Vida e Hooks (4h)

🎯 Objetivos da aula

- Compreender o **ciclo de vida de um componente Vue**.
- Saber **quando** e **onde** executar cada tipo de lógica.
- Utilizar corretamente os **Lifecycle Hooks**.
- Implementar **logs**, **simulação de carregamento** e **efeitos iniciais**.
- Consolidar **boas práticas de mercado**.

---

1) Teoria (aprox. 1h30)

1.1 O que é o Ciclo de Vida

Cada componente Vue passa por etapas bem definidas:

1. Criação
2. Montagem no DOM
3. Atualizações reativas
4. Desmontagem

Os **Lifecycle Hooks** permitem executar código **em momentos específicos** dessas etapas.

1.2 Principais Lifecycle Hooks (Vue 3 – Options API)

🔹 `created()`

- Executado após criação do componente.
- **DOM ainda não existe**.
- Ideal para:
  - Inicializar dados

- Preparar estado
- Chamadas de API (sem depender do DOM)

```
created() { console.log('Componente criado') }
```

🔹 `mounted()`

- Executado quando o componente já está no DOM.
- Ideal para:
  - Acessar elementos HTML
  - Iniciar animações
  - Carregar dados visuais

```
mounted() { console.log('Componente montado no DOM') }
```

🔹 `updated()`

- Executado após atualização reativa.
- Usar com cautela.

```
updated() { console.log('Componente atualizado') }
```

🔹 `unmounted()`

- Executado ao remover o componente do DOM.
- Ideal para:

- Limpar timers
- Remover listeners
- Evitar memory leaks

```
unmounted() { console.log('Componente destruído') }
```

1.3 Boas Práticas de Uso

✔ **created** → inicialização de dados
✔ **mounted** → interação com DOM / UI
✔ **watch / computed** → reatividade
❌ Evitar lógica pesada em `updated`
❌ Não misturar responsabilidades

---

2) Prática (aprox. 2h30) — Ciclo de vida em ação

🎯 **Contexto**
Simular:

- Carregamento inicial
- Logs de ciclo de vida
- Controle de exibição do componente

2.1 `App.vue` — Controle de montagem

```
<template> <main class="container"> <h1>ILP3 – Vue.js Aula 6</h1> <button @click="mostrar = !mostrar"> {{ mostrar ? 'Desmontar' : 'Montar' }} Componente </button> <!-- Montagem condicional --> <UserDashboard v-if="mostrar" /> </main> </template> <script> import UserDashboard from './components/UserDashboard.vue' export default { name: 'App', components: { UserDashboard }, data() { return { mostrar: true } } } </script>
```

2.2 `components/UserDashboard.vue` — Hooks na prática

```
<template> <section class="card"> <h2>Dashboard de Usuários</h2> <!-- Simulação de carregamento --> <p v-if="loading">Carregando dados...</p> <ul v-else> <li v-for="usuario in usuarios" :key="usuario.id"> {{ usuario.nome }} </li> </ul> </section> </template> <script> export default { name: 'UserDashboard', data() { return { usuarios: [], loading: true, timer: null } }, /* ====================== LIFECYCLE HOOKS ====================== */ created() { console.log('[created] Componente criado') }, mounted() { console.log('[mounted] Componente montado no DOM') // Simula chamada de API com delay this.timer = setTimeout(() => { this.usuarios = [ { id: 1, nome: 'Ana' }, { id: 2, nome: 'Bruno' }, { id: 3, nome: 'Carlos' } ] this.loading = false console.log('[mounted] Dados carregados') }, 2000) }, updated() { console.log('[updated] Componente atualizado') }, unmounted() { console.log('[unmounted] Componente desmontado') // Limpeza para evitar memory leak if (this.timer) { clearTimeout(this.timer) } } } </script> <style scoped> .card { border: 1px solid #ddd; padding: 20px; border-radius: 10px; margin-top: 20px; } </style>
```

---

3) O que foi demonstrado (teoria → prática)

✔️ `created`

- Inicialização do componente
- Log estrutural

✔️ `mounted`

- Simulação de API
- Alteração visual

- Controle de loading

✔️ `updated`

- Detecção de reatividade

✔️ `unmounted`

- Limpeza de recursos
- Prevenção de memory leak

---

4) Fluxo visual do ciclo de vida

```
created ↓ mounted ↓ (updated) ← estado muda ↓ unmounted
```

---

5) Atividade prática da aula

🎯 Desafio

1. Adicionar um **contador** que aumenta a cada atualização (`updated`)
2. Exibir quantas vezes o componente foi atualizado
3. Criar um segundo `setInterval` no `mounted`
4. Limpar tudo corretamente no `unmounted`

💡 Dica:

```
updated() { this.atualizacoes++ }
```

---

6) Checklist de encerramento

- Usou lifecycle hooks corretamente
- Entendeu quando usar cada hook
- Simulou carregamento
- Limpou recursos no unmounted
- Evitou lógica fora do ciclo correto